

Scaling Up the Experiment: Before and After

What changed when we moved from a tiny test set to a large one

To scale the experiment up to the full dataset, I split the work into three separate cloud jobs (I call them Kernel A, B, and C). This section explains what each one does and its current status. The rest of the report uses the results from A and B.

1 The three jobs (A, B, C)

Job	What it does	Status
Kernel A	Trains the Boston detector on the full data and measures the untouched pipeline once. This is the setup and timing check.	Done
Kernel B	Runs the main experiment: 15 retraining runs of the detector (varying data size, training length, and seed) and measures the effect on prediction and planning. This produces the headline result.	Done
Kernel C	Repeats the 15-style experiment with a class-balanced training set, to check whether balancing the rare classes still gives the same cases.	Done (7 of 12)

The numbers in this report come from all three jobs. Kernel C finished 7 of its 12 planned runs; the other 5 were the heaviest balancing setting and ran out of cloud time, so I stopped there because the 7 finished runs already answer the question. Its results are in Section 8.

2 The dataset

The experiment now uses the full **nuScenes v1.0 trainval** dataset, the standard public benchmark for this kind of work. It contains 850 scenes: 467 recorded in Boston and 383 in Singapore. We train the detector on Boston and fine-tune it on Singapore, so the retraining is a real geographic domain shift. The dataset (about 45 GB) was already hosted on the cloud platform, so no download or extra compute cost was needed.

The earlier version used only nuScenes-mini, a tiny 10-scene demo subset. That is the core difference: we moved from the demo subset to the full benchmark.

3 The one change that drives everything

The previous version of the experiment used a very small test set: only 3 scenes, which is about 121 camera frames. The new version uses a large test set: 50 scenes, which is 2,020 frames. A scene is a 20-second driving clip and holds roughly 40 frames, so the frame count is the fair way to compare the two.

One point to be clear about: we train on all the Singapore data (229 scenes), but we evaluate on 50 of the 154 Singapore validation scenes, chosen evenly across the set. This keeps the running

time manageable across 15 retraining runs while still giving a test set that is many times larger than before. The 50 scenes are fixed, so every run is measured on exactly the same frames.

Nothing about the method changed. Only the amount of data changed. Every result below follows from that.

4 How much data, before and after

	Before (mini)	After (full trainval)
Boston training scenes	6 scenes	280 scenes, 11,087 images
Singapore fine-tune data	119 images	229 scenes
Test set	3 scenes, 121 frames	50 scenes, 2,020 frames

The test set grew from 121 frames to 2,020 frames, roughly 17 times larger.

5 The pipeline (unchanged)

The system is the same three-part self-driving pipeline:

- **C1, Perception:** a camera detector (YOLOv11) that finds cars, pedestrians, and other agents.
- **C2, Prediction:** predicts where those agents will move.
- **C3, Planning:** decides the car’s driving path.

We retrain only C1 on new-city data and measure what happens to C2 and C3. That part did not change.

6 The real numbers, side by side

Both tables below report the same reference update: the detector is trained on Boston, then fine-tuned on Singapore, and every component is measured on the Singapore test set. “Isolated” means the component is fed perfect ground-truth inputs; “pipeline” means it is fed the live output of the part above it. The gap between the two is the cascade.

The last column reads each row as **better** (green sense: score moved the good way) or **WORSE** (bad way). Entangled enhancement is the pattern where C1 is better, C2 is better, and yet C3 is WORSE at the same time.

Before: mini test set (3 scenes, 121 frames)

Pattern here: **C1 WORSE**, C2 better, **C3 WORSE**. Because the detector itself got worse, this is *not* entangled enhancement (that needs C1 to get better). This is why the mini paper had to add a class-balancing step to force C1 up.

After: full test set (50 scenes, 2,020 frames)

Pattern here: **C1 better, C2 better, and the C3 planning error (L2) WORSE**. This is exactly **entangled enhancement**, and it appears with no trick. The small-data drop in C1 from the mini set is gone.

Part / metric	Before	After	$\Delta\%$	Direction
C1 detector mAP@50	0.1185	0.0644	-45.7	WORSE
C2 pred., isolated	2.09	2.09	0.0	(frozen)
C2 pred., pipeline	15.19	3.95	-74.0	better
C3 plan, isolated	4.93	4.93	0.0	(frozen)
C3 plan, pipeline	6.30	6.58	+4.5	WORSE
C3 collision rate	0.00	0.00	0.0	none

Part / metric	Before	After	$\Delta\%$	Direction
C1 detector mAP@50	0.137	0.256	+86.6	better
C2 pred., isolated	1.12	1.12	0.0	(frozen)
C2 pred., pipeline	7.93	5.76	-27.4	better
C3 plan, isolated	4.64	4.64	0.0	(frozen)
C3 plan, pipeline	5.55	5.67	+2.2	WORSE
C3 collision rate	0.02	0.00	-100	better

One important correction from the earlier reading. The two planning numbers do not agree. On the full test set the collision rate actually **went down**, from 2% to 0%, at the same time as the L2 planning error went **up**. So the detector got better, and the system got safer on collisions, but its path matched the human driver a little less. The two system-level measures move in opposite directions. This is why we do not say the update simply harms the system. We say the terminal metrics disagree, which is the real reason a single number at the output is not enough to accept or reject an update, and why the proposed method reports the whole set of downstream numbers instead of one.

7 The full result: all 15 retraining runs (after)

This is the complete Kernel B result. Each row is one retraining of the detector, under a different setting. The first three columns are the settings we varied: **Data** is how much Singapore data we fine-tuned on (full, half, or a quarter), **Ep.** is how many training epochs, and **Seed** is the random seed (repeating each setting rules out luck).

The three “ Δ ” columns show whether each part improved or worsened compared to the baseline: for C1 a **positive** Δ is better (higher detection score); for C2 and C3 a **negative** Δ is better (lower error). The last column marks the row **YES** when C1 improved and C3 worsened at the same time, which is entangled enhancement. Those rows are in bold.

Baseline (before any retraining): C1 mAP = 0.137, C2 pipeline = 7.93 m, C3 pipeline = 5.55 m.

How to read it in one look: the C1 column is positive in every row (detector always improved), the C2 column is negative in every row (prediction always improved), and the C3 column is positive in 6 rows (planner worse). Those 6 rows are the entangled-enhancement cases: a better detector, a better predictor, and yet a worse driving plan.

Data	Ep.	Seed	C1 Δ (+ better)	C2 Δ (m) (− better)	C3 Δ (m) (− better)	Ent. enh.?
Full	10	1	+0.119	−2.17	+0.124	YES
Full	10	2	+0.122	−2.28	+0.193	YES
Full	10	3	+0.129	−2.15	+0.187	YES
Half	10	1	+0.114	−2.55	−0.121	no
Half	10	2	+0.120	−2.71	−0.230	no
Half	10	3	+0.106	−1.77	−0.151	no
Qtr	10	1	+0.121	−2.71	+0.073	YES
Qtr	10	2	+0.113	−1.29	−0.093	no
Qtr	10	3	+0.101	−2.82	+0.008	YES
Full	5	1	+0.138	−1.08	−0.095	no
Full	5	2	+0.122	−1.88	−0.130	no
Full	5	3	+0.130	−1.08	−0.041	no
Full	20	1	+0.142	−2.59	+0.037	YES
Full	20	2	+0.126	−1.77	−0.083	no
Full	20	3	+0.130	−2.44	−0.020	no
Totals			15/15 better	15/15 better	6/15 worse	6 YES

8 The class-balanced check (Kernel C)

In the old small-scale experiment, we had to balance the rare classes (oversample the images that contain trucks, buses, and so on) just to make the detector score go up at all. On the full data that step is no longer needed, because Kernel B already shows the effect. So here Kernel C asks a different question: if we still balance the classes, does the entangled enhancement survive, or was it an artifact of the class imbalance?

I ran two balancing strengths, two training lengths, and three seeds (12 planned runs). Seven finished; the other five were the heaviest setting and ran out of cloud time. The seven finished runs are enough to answer the question.

Balance	Ep.	Seed	C1 Δ (+ better)	C3 Δ (m) (− better)	Ent. enh.?
Light ($p=0.5$)	10	1	+0.127	−0.021	no
Light ($p=0.5$)	10	2	+0.122	−0.023	no
Light ($p=0.5$)	10	3	+0.112	+0.014	YES
Light ($p=0.5$)	20	1	+0.141	+0.139	YES
Light ($p=0.5$)	20	2	+0.143	+0.094	YES
Light ($p=0.5$)	20	3	+0.137	−0.001	no
Heavy ($p=1.0$)	20	1	+0.125	+0.075	YES
Totals			7/7 better	4/7 worse	4 YES

The result: even with class balancing, the detector still improves in every run (7 of 7), and entangled enhancement still appears (4 of 7). So the effect is not caused by class imbalance. It is a real property of the pipeline under retraining, and balancing the classes does not remove it.

9 Re-checking CARA’s cheap early-warning screen

The proposed method has a cheap first-pass screen: instead of running the whole pipeline, it looks only at how much the new detector’s outputs drifted from the old one, and uses that drift score to guess which updates are risky. On the small test set this screen looked promising. On the full test set I re-checked it properly, and the honest answer is more limited.

What we asked of the drift score	Before (mini)	After (full)
Does it track <i>how much</i> the plan moves? (rank correlation)	about 0.41	about 0.56
Does it tell a worse update from a safe one? (AUROC)	about 0.70	about 0.48 (chance)

So the drift screen does track the *size* of the disturbance, and a little better than before (0.56). But it cannot tell the *direction*: it does not separate the updates that hurt the planning metric from the ones that help. On the full data all the drift scores fall in a narrow band, so the risky and safe updates sit mixed together. The earlier 0.70 was small-sample optimism.

I also validated it the careful way this time, choosing the threshold on one half of the updates and testing on the untouched other half (8 updates). On that held-out test it still catches every planning-metric regression, but with low precision, and with only 8 updates the error bars are wide (for example the recall could sit anywhere from 0.44 to 1.00). The takeaway is honest and useful: the cheap screen is a good way to *prioritize* which updates to check first, but it is not a stand-alone accept or reject rule. The full check that runs the whole pipeline remains the reliable one.

10 Summary of what changed

What we measured	Before (3 sc., 121 fr.)	After (50 sc., 2,020 fr.)
Detector improves after retraining?	0 of 15	15 of 15
Prediction improves?	mixed	15 of 15
Entangled enhancement seen directly?	0 of 15 (needed a trick)	6 of 15 (no trick)
Baseline collision rate	0%	2%
Collision rate after retraining	0%	0% (fell from 2%)
Plan shift range	1.33 to 7.80 m	0.78 to 1.50 m

The plan-shift range also became tighter, from 1.33–7.80 m down to 0.78–1.50 m. This is expected: with a much larger test set the measurement is less noisy, so the extreme values from the tiny 3-scene set do not appear.

11 Baseline system, before any retraining

For reference, the untouched pipeline measured on the 2,020-frame test set:

Metric	Value	Reading
C1 detector mAP	0.137	detector works
C2 prediction, isolated	1.12 m	clean prediction
C2 prediction, pipeline	7.93 m	error inherited from the detector
C3 planning, isolated	4.64 m	planner alone
C3 planning, pipeline	5.55 m	worse than isolated
C3 collision rate	2%	collisions now occur

The planner error in the pipeline (5.55 m) is larger than the planner error in isolation (4.64 m). That gap is the cascade: the planner inherits the error that flows down from perception. It now holds across 50 scenes instead of 3.

12 What this means for the paper

1. The main claim is stronger. Entangled enhancement in a real driving pipeline is now shown by 6 ordinary retraining runs on full data, rather than by a special class-balancing step on 3 scenes.
2. One old finding is removed. The “detector score drops after retraining” result was caused by the small 119-image fine-tune set. With full data it does not happen, so that finding is dropped.
3. A new and more careful result appears. At full scale the baseline pipeline has a 2% collision rate (there were none on 3 scenes), and retraining drives it to zero while the L2 planning error goes up. The two system-level numbers move in opposite directions, so the finding is stated as a disagreement between metrics, not as a plain harm. This makes the point of the proposed method clearer: it must report the whole set of downstream numbers, because no single one is safe to trust alone.
4. The cheap early-warning screen was re-checked honestly. It tracks the size of the disturbance but not its direction, so it is a way to prioritize which updates to check, not a stand-alone accept or reject rule.

13 What is left

All three cloud jobs are finished, so the experiment itself is complete and every number in this report is final. Two small things remain on the writing side, and neither needs new experiments to state:

- The five class-balanced runs that ran out of cloud time were the heaviest setting. The seven finished runs already show the effect survives balancing, so finishing the last five is optional and would only add more of the same.
- One check would strengthen the smallest of the six entangled-enhancement cases (the one where the planning error rose by only 0.008 m): running the unchanged detector against itself to measure the natural measurement noise. If that noise is below 0.008 m, all six cases stand; if not, that one case is borderline. This is a quick run I can do next.